

PRACTICAL_07

1. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

CODE:

```
root@SRIVALLI: ~  
GNU nano 7.2 memory_layout.c *  
#include <stdio.h>  
#include <stdlib.h>  
  
int global_var = 100;  
static int static_var = 200;  
  
void display()  
{  
    printf("Code Segment Address    : %p\n", display);  
}  
  
int main()  
{  
    int stack_var = 300;  
  
    int *heap_var = (int *)malloc(sizeof(int));  
    *heap_var = 400;  
  
    display();  
  
    printf("Global Variable Address: %p\n", &global_var);  
    printf("Static Variable Address: %p\n", &static_var);  
    printf("Heap Variable Address  : %p\n", heap_var);  
    printf("Stack Variable Address : %p\n", &stack_var);  
  
    free(heap_var);  
  
    return 0;  
}
```

RESULT:

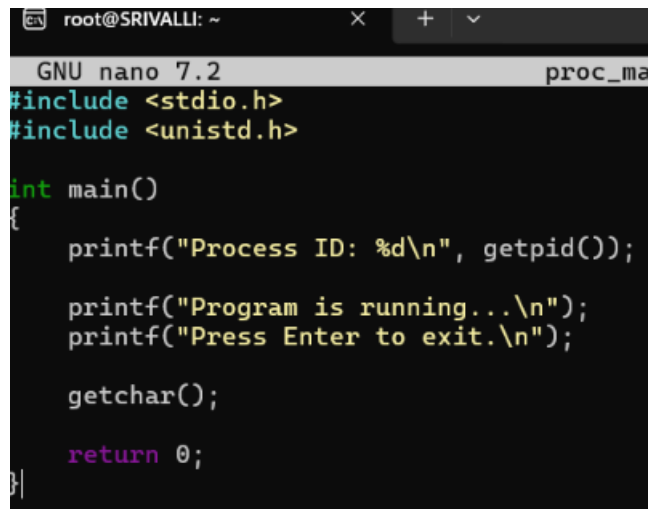
```
root@SRIVALLI:~# nano memory_layout.c  
root@SRIVALLI:~# gcc memory_layout.c -o memory_layout  
root@SRIVALLI:~# ./memory_layout  
Code Segment Address    : 0x5fb3a74581a9  
Global Variable Address: 0x5fb3a745b010  
Static Variable Address: 0x5fb3a745b014  
Heap Variable Address  : 0x5fb3d051e2a0  
Stack Variable Address : 0x7ffd50e9429c
```

REPORT:

In this experiment, a program was developed to display the addresses of the code, global, static, heap, and stack memory segments in a Linux process. By observing these addresses, the process memory layout can be understood. This experiment helps in learning how different types of data are stored and managed in memory during program execution.

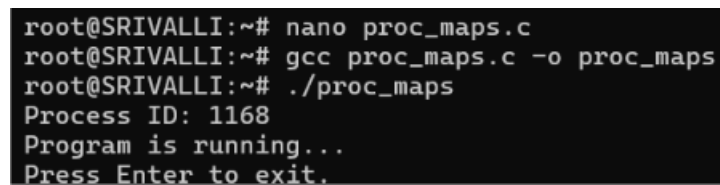
2. Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

CODE:



```
root@SRIVALLI: ~  
GNU nano 7.2 proc_ma  
#include <stdio.h>  
#include <unistd.h>  
  
int main()  
{  
    printf("Process ID: %d\n", getpid());  
  
    printf("Program is running...\n");  
    printf("Press Enter to exit.\n");  
  
    getchar();  
  
    return 0;  
}
```

RESULT:



```
root@SRIVALLI:~# nano proc_maps.c  
root@SRIVALLI:~# gcc proc_maps.c -o proc_maps  
root@SRIVALLI:~# ./proc_maps  
Process ID: 1168  
Program is running...  
Press Enter to exit.
```

REPORT:

In this experiment, the `/proc/PID/maps` file was used to view the memory organization of a running process. It shows different memory regions such as the code segment, data segment, heap, and stack. This helps in understanding how memory is allocated and managed in a Linux process.